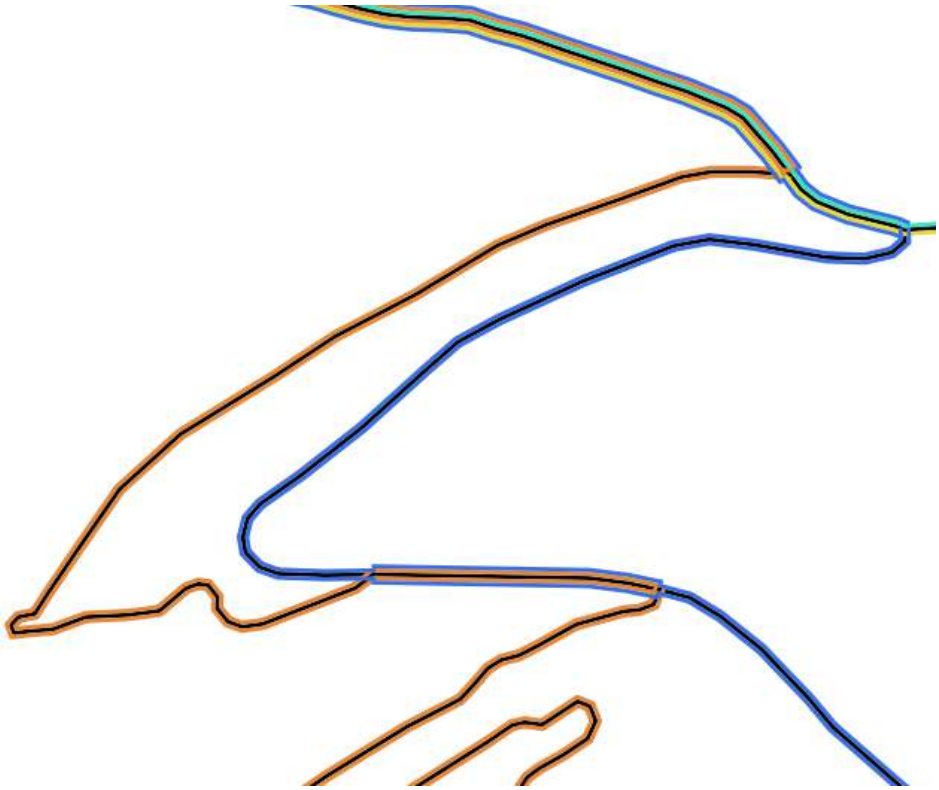
Géodata Paris

Université Gustave Eiffel

Rapport de projet

Cycle des ingénieurs de l'ENSG 3ème année et M2 IGAST

Implémentation d'un plugin QGIS permettant de représenter des itinéraires « à la manière de plans de bus »



Titouan COURGEY, Nada ED-DARIY, Sarah FORCIERI et Augustin-Pierre HOSTEIN

Mars 2026

Résumé

Ce projet s'inscrit dans le cadre du module d'analyse spatiale et porte sur le développement d'un plugin QGIS qui est destiné à représenter des itinéraires ou portions d'intérêt selon un principe inspiré des plans de transport en commun. Appliqué à treize itinéraires situés dans le massif du Vercors, le plugin vise à résoudre les problèmes de superposition des tracés en appliquant automatiquement un décalage latéral lorsque plusieurs itinéraires partagent un même tronçon. Cette approche permet d'améliorer à la fois la lisibilité de la carte et de distinguer clairement chaque parcours.
Le plugin a été développé en Python en s'appuyant sur les bibliothèques PyQGIS et PyQt5. Il intègre une interface graphique interactive facilitant la sélection des données, l'exécution du traitement et la visualisation des résultats au sein de QGIS.

Mots clés : QGIS, plugin, décalage, strokes, cartographie, visualisation.

Table des matières

Introduction **5**
0.1 Contexte . . . 5
0.2 Objectifs . . . 6
0.3 Plan du rapport . . . 6

1 État de l'art **7**
1.1 Règles de design . . . 7
1.2 Méthodes et modèles existants pour la représentation des réseaux . . . 8
1.3 Représentation géographiquement contrainte des réseaux de transport . 10
1.4 Architecture de la donnée . . . 11

2 Présentation des données **13**
2.1 Zone d'étude . . . 13
2.2 Présentation des données test . . . 14

3 Méthodologie **16**
3.1 Création et visualisation des strokes routiers et d'itinéraires . . . 16
3.1.1 Définition et intérêt des strokes . . . 16
3.1.2 Création des strokes routiers . . . 16
3.1.3 Création des strokes d'itinéraire . . . 17
3.1.4 Visualisation des itinéraires . . . 19
3.2 Outil de sélection des itinéraires . . . 19
3.3 Outil d'échange des itinéraires . . . 20
3.4 Génération des croisements . . . 21

4 Mise en œuvre et résultats **22**
4.1 Mise en œuvre des méthodes . . . 22
4.1.1 Création des strokes . . . 22
4.1.2 Attribution d'un offset pour chaque itinéraire . . . 24
4.1.3 Outil de sélection . . . 24
4.1.4 Outil d'échange . . . 25
4.1.5 Génération des croisements . . . 26
4.2 Interface graphique et fonctionnalités . . . 27
4.3 Limites et perspectives . . . 31
4.3.1 Limites et difficultés rencontrées . . . 31
4.3.2 Perspectives d'amélioration . . . 32

Conclusion **34**

Introduction

0.1 Contexte

Les pratiques « outdoor » connaissent aujourd'hui un développement important et s'appuient fortement sur l'utilisation de supports cartographiques pour orienter les pratiquants. Toutefois, la représentation cartographique des réseaux d'itinéraires peut devenir complexe lorsque plusieurs itinéraires empruntent des tronçons communs. Dans ce cas, la cartographie classique atteint rapidement ses limites : les tracés se superposent et les informations se confondent. Pour cela, la représentation des itinéraires multiples à la manière des plans de bus (cf Figure 1) permet de garder une bonne lisibilité sans réduire la compréhension du réseau. Cependant, ce type de représentation est produit aujourd'hui manuellement sur des logiciels de design, comme Illustrator, car les SIG intègrent très mal la gestion de ces itinéraires.

FIGURE 1 – Exemple d'un plan de bus : Plan du réseau de bus RATP de Paris (Source : RATP)

Dans ce contexte, un plugin QGIS a été développé pour permettre une meilleure gestion des itinéraires multiples. Ce plugin permet à l'utilisateur de gérer directement le rendu graphique (couleur et épaisseur) des tracés des itinéraires ainsi que leur placement les uns par rapport aux autres.

0.2 Objectifs

Ce projet répond à un besoin concret : avoir un outil interactif pour gérer et visualiser plusieurs itinéraires directement dans un environnement SIG. Pour cela, trois objectifs ont été définis :

1. Créer un modèle de données pour relier les tronçons de routes et de sentiers aux itinéraires qui les empruntent.
2. Mettre en place ce modèle dans QGIS, ainsi que les méthodes pour le construire à partir de deux couches vecteur fournies en entrée.
3. Définir et développer des méthodes pour dessiner le réseau et les itinéraires décalés, en faisant attention aux cas complexes comme la gestion des carrefours.

0.3 Plan du rapport

Pour mener à bien ce projet, nous nous sommes d'abord approprié théoriquement les problématiques liées à ce sujet par un état de l'art bibliographique (partie 1). Après une présentation des données test sur lesquelles nous travaillons (partie 2) nous présentons la méthodologie suivie pour répondre au sujet (partie 3). Enfin, nous expliquons comment nous avons mis en œuvre cette méthodologie afin d'aboutir à la création d'un Plugin (partie 4).

État de l'art

Chapitre 1

Pour nous aiguiller dans notre projet, nous avons réalisé un état de l'art en introduction à nos travaux. Cela nous a permis de nous informer quant à l'avancée des recherches sur le sujet, de prendre connaissance des axes existants et des potentielles difficultés que nous allons rencontrer.
Pour cela, 4 axes de travail ont été choisi :

- Les règles de design, nous permettant de prendre connaissance des règles que nous devrons appliquer afin de respecter la lisibilité et la compréhension des réseaux ;
- Les méthodes et modèles existant pour la représentation de réseaux, afin de nous informer sur l'étendue des recherches dans ce domaine et potentiellement sur d'autres solutions existantes ;
- La représentation géographiquement contrainte des réseaux de transport, pour nous informer des potentielles difficultés que nous allons rencontrer lors du projet, et s'il existe des solutions ;
- L'architecture de la donnée, nous permettant de nous informer sur les données usuellement utilisées ou nouvellement développées dans ce genre de sujets, pour partir sur des bases cohérentes et à jour.

1.1 Règles de design

L'article de Teulade-Denantes, Maudet et Duchêne, 2015 constitue la référence centrale de notre projet : il présente une méthode pour automatiser la représentation cartographique de plusieurs itinéraires superposés sur un même tronçon de route physique à partir de données vectorielles. Pour cela, les auteurs proposent de décaler latéralement les itinéraires de part et d'autre de l'axe central (la route) de façon à ce que chaque itinéraire soit visible et distinct. Chaque itinéraire est distingué par sa couleur, et est modélisé de sorte à être perçu comme un tout. Cette méthode est reprise pour la représentation de nos itinéraires.
La méthode proposée repose sur un problème de satisfaction d'optimisation sous contraintes (COP) qui appartient au paradigme du problème de satisfactions des contraintes (CSP). Ce dernier problème est défini formellement par un triplet de variables, domaines et contraintes. Deux classes de méthodes sont proposées : des algorithmes systématiques, qui garantissent de trouver la solution optimale si elle existe, et des algorithmes non systématiques qui approximent la solution optimale si elle existe.
Les données vectorielles utilisées par les auteurs sont un réseau routier fragmenté en segments de routes. Un segment est défini comme une polyligne entre deux croisements de routes. Pour éviter que les segments ne se déplacent trop les uns par rapport aux autres, les segments formant une ligne visuelle continue sont regroupés en « strokes ».

Les strokes sont au cœur de ce projet, la partie 3.1 détaille comment ils sont créés et visualisés avec nos données. Ensuite, les auteurs définissent trois critères pour assurer une continuité visuelle : l'angle formé par les tronçons au croisement doit être inférieur à un seuil, la même symbologie doit être gardée et les segments regroupés doivent appartenir au même itinéraire. Les auteurs définissent également des règles de lisibilité, basées sur la théorie de la Gestalt sur la perception visuelle, et cherchent la configuration qui minimise les « mauvais » placements (croisements inutiles ou discontinuités). 4 contraintes sont identifiées :

— C1 ou contrainte de proximité : tous les segments d'itinéraire portés par le même tronçon de route doivent être contigus les uns aux autres ainsi qu'au tronçon de route, il ne doit pas y avoir de trous/d'espace vide entre deux segments ;
— C2 ou contrainte de continuité : une ligne doit, autant que possible, rester du même côté de la route tout au long de son trajet pour faciliter le suivi visuel par l'utilisateur ;
— C3 ou minimisation des croisements entre segments : les croisements de routes doivent être évités le plus possible ;
— C4 ou minimisation des regroupements de croisements : avoir un seul point où tout se croise est visuellement préférable à une multitude de petits croisements dispersés sur une même intersection.

Ces règles sont intéressantes d'un point de vue représentation du réseau même si l'optimisation des placements ne fait pas partie du projet. Elles ont guidé l'implémentation du plugin : la contrainte de proximité est respectée lors de l'application de l'offset qui se fait sans laisser d'espaces entre les rubans, et la contrainte de continuité est également assurée.
Les auteurs soulignent la difficulté autour de la gestion des croisements de routes à cause du chevauchement des symboles. C'est une des difficultés majeures rencontrées lors du projet, et c'est l'objet des parties 3.4 et 4.3. De plus, le plugin développé met en œuvre une des pistes d'améliorations proposées par les auteurs : la gestion des couleurs des itinéraires (parties 3.1.4 et **??**).
Cet article est un élément central dans l'approche de notre projet. Il définit un cadre formel de règles de design et de modélisation à mettre en place pour la représentation des itinéraires.

1.2 Méthodes et modèles existants pour la représentation des réseaux

La représentation des réseaux de transport est un sujet important en cartographie. Pour mieux comprendre les méthodes existantes et choisir une solution adaptée, nous avons étudié les travaux de Nöllenburg, 2014 et Nöllenburg et Wolff, 2011.

L'article de Nöllenburg, 2014 constitue une référence complète sur les méthodes de génération automatique de plans de métro. L'auteur explique que le problème peut être décrit comme suit : on part d'un graphe $G = (V, E)$ représentant le réseau, où les sommets V représentent les stations et les arêtes E les connexions entre elles, ainsi qu'un ensemble de lignes L couvrant ce graphe.
L'objectif est de produire un plan simplifié qui conserve la structure du réseau, respecte certaines règles de dessin et offre un résultat clair et lisible pour l'utilisateur.

L'article présente dix règles de design importantes utilisées dans les algorithmes de génération de plans de métro. Par exemple, il est essentiel de ne pas modifier la topologie du réseau, de limiter les orientations des arêtes à des directions octilinéaires (horizontales, verticales et à $\pm 45°$), et de tracer les lignes de la manière la plus droite possible.
Il est également nécessaire de préserver les positions relatives des stations, d'utiliser des longueurs de segments similaires, d'éviter le chevauchement des étiquettes, et de maintenir une distance minimale entre les éléments qui ne sont pas liés.
Nöllenburg, 2014 classe les méthodes existantes en quatre familles :

- Les méthodes d'optimisation globale : notamment la programmation linéaire mixte (*Mixed Integer Programming (MIP)*), qui permettent d'obtenir une solution optimale, mais avec un temps de calcul élevé ;
- Les méthodes basées sur les forces : elles utilisent un modèle physique où les stations se repoussent comme des particules et les connexions agissent comme des ressorts. Cette approche est rapide, mais ne garantit pas le respect strict des contraintes géométriques ;
- Les méthodes d'optimisation locale : elles améliorent progressivement une disposition initiale en déplaçant les sommets sur une grille, afin de minimiser une fonction de coût multicritère ;
- Les méthodes spécialisées : elles se concentrent sur des problèmes précis, comme la minimisation des croisements entre lignes parallèles ou le placement des étiquettes.

Cette classification nous a permis de situer notre approche : le plugin Itinéraires Décalés utilise une optimisation locale, en calculant les décalages (offsets) segment par segment sans chercher une solution globale optimale ce qui garantit des temps de calcul compatibles avec une utilisation interactive dans QGIS.

Nöllenburg et Wolff, 2011 quant à eux, proposent la première solution formelle complète pour générer automatiquement des plans de métro, en combinant la mise en page et l'étiquetage dans un seul modèle de programmation linéaire mixte (*MIP*).
Les auteurs distinguent deux types de contraintes :

- **Les contraintes strictes (*Hard Constraints*)** : elles doivent être respectées obligatoirement. Par exemple, conserver l'ordre des connexions autour de chaque station afin de préserver la structure du réseau, ou éviter les croisements non souhaités.
- **Les contraintes souples (*Soft Constraints*)** : elles sont optimisées pour améliorer l'aspect visuel du plan. Par exemple, réduire le nombre d'angles pour obtenir des lignes plus droites, ou maintenir des longueurs de segments similaires.

La fonction objectif s'écrit :

$$\min \ \lambda(S_1) \cdot \mathrm{cost}(S_1) + \lambda(S_2) \cdot \mathrm{cost}(S_2) + \lambda(S_3) \cdot \mathrm{cost}(S_3),$$

où les poids λ permettent d'ajuster l'importance de chaque critère selon les besoins.
Pour résoudre ce problème, les auteurs utilisent un système de coordonnées étendu (x, y, z_1, z_2), qui facilite la gestion des différentes orientations des lignes. Ils introduisent également une distance $L - infini$ afin de mesurer l'espacement entre les éléments.
La méthode a été testée sur plusieurs réseaux, notamment celui de Londres. Les résultats montrent une très bonne qualité visuelle, proche des cartes officielles. Cependant, le

temps de calcul peut être élevé, pouvant atteindre jusqu'à 10 heures selon la taille du réseau.
Ces travaux là ont influencé notre implémentation sur deux aspects principaux. D'une part, la distinction entre contraintes strictes et contraintes souples nous a guidés dans la conception : dans `style.py`, l'absence de chevauchement entre les rubans correspond à une contrainte stricte, tandis que la répartition symétrique des offsets est traitée comme une optimisation locale.
D'autre part, ces travaux confirment que la gestion de plusieurs lignes sur un même réseau constitue un problème complexe, généralement considéré comme *NP-difficile*. Cela justifie le choix d'une approche heuristique dans notre implémentation, plutôt qu'une recherche de solution optimale globale, souvent trop coûteuse en temps de calcul.

1.3 Représentation géographiquement contrainte des réseaux de transport

Un article également très intéressant dans le cadre de notre étude et que nous avons étudié est celui de Bast, Brosi et Storandt, 2019, qui propose une méthode d'optimisation et de génération des itinéraires décalés et des croisements, qui constitue une problématique majeure de notre travail. Il propose également une solution, a priori accessible et open source, permettant de calculer ces paramètres d'optimisation et de génération de tracés. Enfin, l'article vise à produire un réseau de transport non seulement topologique, mais surtout géographiquement fidèle, avec des itinéraires respectant le tracé réel des routes et voies existantes.
L'étude en question présente LOOM (Line Ordering Optimized Maps), un générateur automatique de cartes de transport géographiquement précises. L'objectif de LOOM est de produire des cartes de réseaux de transports, à partir de données standardisées (format GTFS). L'originalité de l'étude réside dans la fidélité géographique du réseau (alors que la plupart des cartes réseaux sont uniquement topologiques), la minimisation des croisements et des séparations de lignes entre elles, et le tout dans un rendu clair et lisible. L'ensemble du processus est structuré en trois étapes principales : construction du graphe des lignes, optimisation de l'ordre des lignes, et rendu final.
La première étape consiste à transformer les trajets en un graphe des lignes, où chaque station est représentée par un nœud et chaque segment partagé par un ensemble de lignes devient une arête unique. Pour garantir la précision géographique, deux segments sont considérés comme identiques si leur distance est inférieure à un seuil. Les segments partagés sont fusionnés et de nouveaux nœuds "non-station" sont introduits aux points de séparation, permettant de reproduire exactement le tracé réel des lignes en simplifiant le graphe.
La deuxième étape vise à minimiser les croisements et les séparations de lignes (MLNCM et MLNCM-S) sur le graphe. LOOM utilise une formulation via programmation linéaire en nombres entiers (ILP pour Integer Linear Programming) pour déterminer l'ordre optimal des lignes. 3 versions d'algorithmes d'optimisation sont développées, chacune ayant plus ou moins de contraintes :

- ILP de base (Baseline ILP) : encode les positions des lignes et les croisements éventuels sur chaque arête, avec un nombre important de contraintes ;
- Formulation améliorée (Improved ILP Formulation) réduit les contraintes et ajoute la pénalité pour la séparation des lignes partenaires (MLNCM-S), afin que les

lignes partageant un parcours commun restent côte à côte ;
— Formulation améliorée avec pénalités de séparation (Improved ILP with added separation penalty), où des facteurs de pondération permettent de préférer ou d'éviter les croisements et séparations dans certaines zones, en considérant par exemple les stations.

Le rendu final se base sur le graphe optimisé et l'ordre des lignes. Chaque ligne est déplacée perpendiculairement sur son segment selon sa position, garantissant un espacement régulier sans déformer le tracé réel. Les nœuds sont élargis pour permettre la connexion des lignes, en conservant leur position exacte dans l'espace. Les stations sont représentées par des polygones ou rectangles pivotés.
LOOM a été évalué sur les réseaux de six villes, les résultats montrent que l'ILP amélioré atteint la solution optimale en moins de 1,5 seconde pour les jeux de données moyens, et l'ILP (simple) en moins d'une minute. La réduction du graphe initial a permis de diminuer en moyenne de plus de 50% le nombre de lignes et de colonnes dans les ILP, améliorant significativement les temps de calcul. Enfin, la pondération concernant les séparations dans les stations produit des cartes visuellement plus agréables.

1.4 Architecture de la donnée

Cette section réalise une synthèse bibliographique de l'article Xiao et al., 2024.

Cet article aborde la problématique de la généralisation et nous présente un nouvel outil de généralisation de réseaux routiers utilisant des réseaux convolutifs de graphes et des Mesh-Line Structure Unit (MLSU). Il a été publié le 10 octobre 2024. L'intérêt premier de cette technique vis à vis des techniques traditionnelles de généralisation est qu'elle utilise le deep learning afin de franchir les limites dont souffrent ces méthodes : l'utilisation d'un réseau convolutif de graphes permet de s'affranchir de l'expertise humaine dont dépendent fortement les méthodes de généralisation.
Un MLSU est une unité de description enrichie qui est associée à chaque segment routier. Chaque segment possède deux MLSU : un par face la bordant. Le but de cette unité de description est d'avoir le maximum d'informations concernant sa géométrie et ses interactions topologiques de façon à ce que le réseau de neurones ait le plus de détails possibles concernant le réseau en chaque point. Mieux on informe le réseau de neurones, plus ses choix de généralisation seront justes.
En plus de l'intégration des MLSU, une des principales caractéristiques de leur approche est de considérer les MLSU comme les nœuds du graphe. Les arêtes du graphe représentent donc chaque interaction entre deux MLSU, et deux MLSU sont en interaction dès qu'ils se touchent. L'article justifie le choix de représenter les MLSU comme des nœuds car cela permet de centrer la décision de généralisation sur les objets effectivement supprimés ou conservés.
Le réseau convolutif de graphes est un réseau de neurones composé de plusieurs couches de convolution sur graphe. Chaque couche met à jour la représentation d'un nœud en agrégeant les caractéristiques du nœud considéré à celles des nœuds voisins. En sortie du réseau, l'algorithme ressort une valeur valant 0 ou 1 pour chaque MLSU ; on a donc deux valeurs par segment routier. Ces deux valeurs passent ensuite dans une porte logique ET : si la valeur finale est 0 le segment est gardé, et si elle vaut 1 le segment est supprimé. La fonction de perte de l'algorithme pénalise les erreurs de classification : les

suppressions de segments qui devraient être conservés, et les conservations de segments qui devraient être supprimés.
Lors de la suppression d'un segment, les deux faces sont fusionnées, entraînant une fusion des MLSU. La façon dont a été construit le graphe à l'origine, définissant les interactions entre les faces, permet à l'algorithme d'être très rapide dans la fusion nouvelle des faces.
Pour les besoins de la généralisation, le MLSU possède plusieurs attributs différents selon s'il concerne la route, la face, ou l'interaction de la route et de la face. Tous ces attributs permettent de renseigner au mieux le réseau de neurones sur la nature de la route. 22 attributs concernent la route (importance, degré, aire de Voronoï, longueur du stroke ...), 5 concernent la face (aire, nombre de routes, orientation du plus petit rectangle englobant...) et 3 caractérisent l'interaction entre la route et la face (différence d'orientation des plus petits rectangles englobant, rapport entre la longueur de la route et celle du périmètre de la face...)
Pour mettre en pratique le modèle, les chercheurs ont entraîné leur algorithme sur les états du Kentucky, Tennessee, Caroline du nord, Caroline du sud et Géorgie, avant de le tester sur les états de l'Oklahoma, l'Arkansas et la Louisiane. Les objets utilisés sont les routes issues de l'USDA (US Department of Agriculture) à l'échelle 1 :1000000 et 1 :2000000. Les résultats obtenus sont très prometteurs et se rapprochent très bien des visualisations établies précédemment par les cartographes. La précision globale obtenue est de 81%. Comparée aux méthodes traditionnelles basées sur des règles (routes ou faces) ou aux méthodes de deep learning non adaptées aux graphes, cette méthode semble mieux préserver la connectivité globale du réseau, respecte davantage la hiérarchie routière, évite les suppressions incohérentes de segments structurants et produit des cartes plus lisibles d'un point de vue structurel.
En dépit des ces avantages, cette méthode nécessite de disposer de données multi-échelles de qualité, possède un coût computationnel plus élevé que les méthodes classiques et centre la généralisation sur la sélection de segments sans traitement de la simplification géométrique.
L'intérêt de cet article vis à vis de notre problématique est la nouvelle structure d'entité : solution qu'il introduit au travers du Mesh-Line Structure Unit (MLSU). Cette nouvelle entité peut être pertinente à intégrer dans notre projet pour illustrer les interactions entre les parties du graphe et faciliter les choix de placement de ligne suivant l'interaction des géométries entre elles. La connaissance de cette nouvelle unité de description nous permet de poser un regard différent sur le problème nous conduisant à imaginer la problématique sous un autre angle afin de trouver des réponses qui diffèrent de l'habitude.

Présentation des données

Chapitre 2

2.1 Zone d'étude

Le projet s'appuie sur un ensemble d'itinéraires de randonnée situés dans le massif du Vercors, plus précisément dans le secteur des deux soeurs et de la Grande Moucherolle, au dessus de Villard-de-Lans, Corrençon-en-Vercors et Château-Bernard. Ce territoire possède un réseau de sentiers important, où plusieurs itinéraires passent par les mêmes tronçons, ce qui en fait un cas d'étude pertinent pour analyser notre problématique de représentation et de visualisation des itinéraires dans un contexte de superposition des parcours.

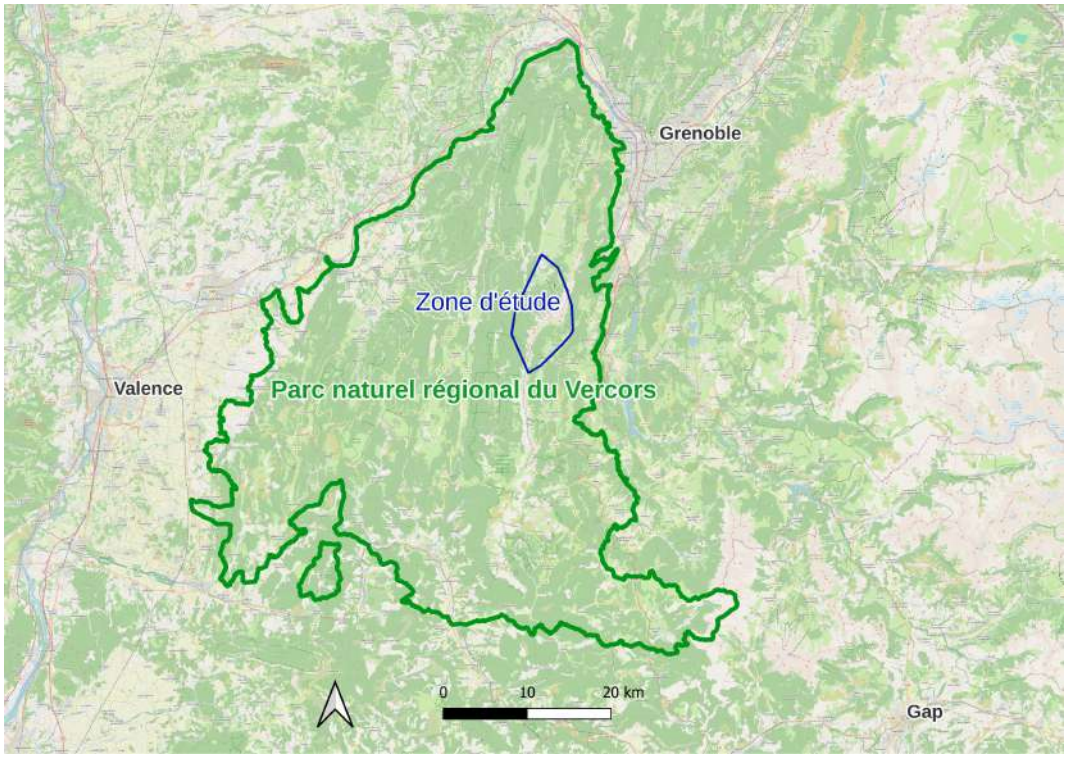


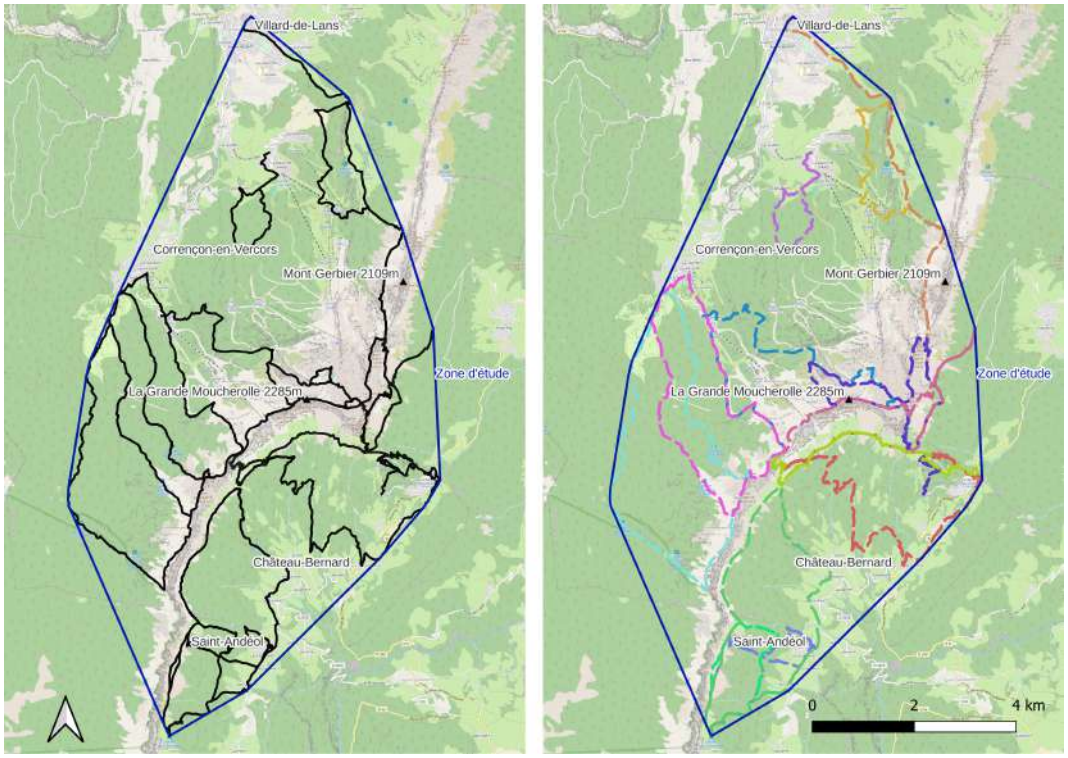
Figure 2.1 – Localisation de la zone d'étude dans le massif du Vercors.

Les données utilisées contiennent 613 tronçons de sentiers, répartis sur 13 itinéraires de randonnée distincts, constituant un jeu de données représentatif de la complexité du réseau de chemins dans cette zone.

2.2 Présentation des données test

Les données fournies permettent de tester les codes réalisés au fur et à mesure de leur avancement. Elles décrivent à la fois la géométrie du réseau de sentiers et la structure des itinéraires.
Le fichier `reseau.shp` est une couche vecteur de type Multilinestring qui représente les tronçons élémentaires du réseau de sentiers. Chaque tronçon est identifié par l'attribut `cleabs`, qui constitue un identifiant unique issu du référentiel de l'IGN. Cette couche permet de représenter la géométrie du réseau de sentiers.
Le fichier `itineraire.shp` est une couche vecteur de type Multilinestring représentant les 13 itinéraires de randonnée. Chaque entité correspond au tracé complet d'un itinéraire, construit à partir des tronçons du réseau décrits dans le fichier reseau.shp.
Le fichier `ITI_TRONCON_ORDRE_SENS_GEOM.csv` constitue une table de correspondance reliant les itinéraires aux tronçons du réseau. Il contient :

- L'identifiant de l'itinéraire (`r.id`) ;
- Le nom de l'itinéraire (`r.title`) ;
- L'ordre de parcours des tronçons (`f.order`) ;
- L'identifiant du tronçon (`t.cleabs`), utilisé comme clé de jointure avec `reseau.shp` ;
- Le sens de parcours (`n.direction`), pouvant prendre les valeurs `same`, `inverse` ou `null`.



(a) Contenu de la couche reseau.shp (b) Contenu de la couche itineraire.shp

FIGURE 2.2 – Visualisation de l'emprise des données dans la zone d'étude

L'utilisation de ces trois fichiers permet de relier les itinéraires aux tronçons du réseau et de connaître quels itinéraires empruntent chaque tronçon.

Le fichier CSV pouvant être facilement généré à partir du fichier shapefile des itinéraires, et inversement, nous avons décider que l'algorithme ne prendrait en entrée que le fichier shapefile.

Méthodologie

Chapitre 3

3.1 Création et visualisation des strokes routiers et d'itinéraires

3.1.1 Définition et intérêt des strokes

Notion introduite par Thomson et Richardson, 1999, un stroke est la succession de tronçons connectés entre eux et qui semblent être tracés d'un même trait de crayon. Autrement dit, il s'agit de considérer comme une seule entité les tronçons qui se suivent et disposent des mêmes caractéristiques qualitatives.

Dans ce projet, nous avons choisi de travailler par strokes car nous souhaitons raisonner à un niveau supérieur à celui d'un tronçon. Comme expliqué dans Teulade-Denantes, Maudet et Duchêne, 2015, la considération de strokes dans ce problème revient à considérer le placement des itinéraires au-delà des tronçons ce qui simplifie grandement le problème. La notion de stroke nous semble beaucoup plus cohérente vis à vis de notre projet que celle des Mesh Line Structure Unit développée dans le papier de Xiao et al., 2024 car la prise en compte des faces ne nous semble pas pour l'instant pertinente et celle-ci surcharge la donnée.

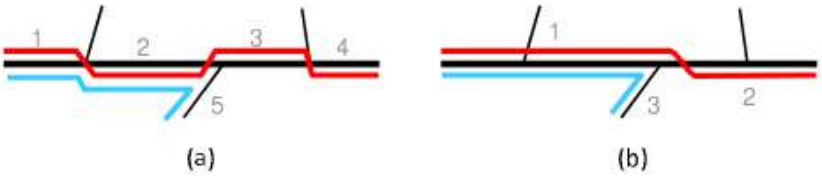


Figure 3.1 – (a) Modélisation initiale : 7 strokes de route et 7 segments d'itinéraire ; (b) Segments d'itinéraires regroupés en strokes : 4 strokes d'itinéraire créés à partir des 7 segments. Source : Teulade-Denantes, Maudet et Duchêne, 2015

3.1.2 Création des strokes routiers

La création des strokes routiers revient à simplifier le graphe routier en ne gardant que les nœuds de degré supérieur ou égal à 3 et en réduisant à 1 le nombre d'entités par arc du graphe. La figure ci-dessous montre une représentation colorée des strokes qui met en évidence le découpage.

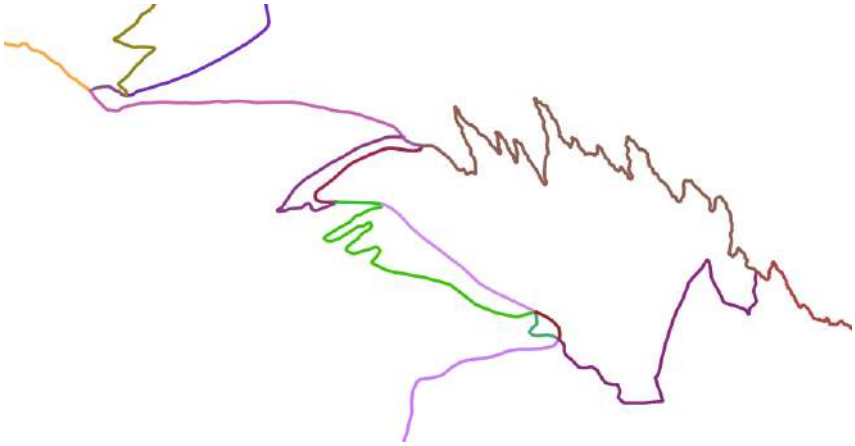


FIGURE 3.2 – Visualisation des strokes routiers : une couleur par stroke.

Les strokes routiers sont créés à partir de la couche réseau. Les strokes possèdes les mêmes attributs et le même type de géométrie que les éléments du réseau. Chaque stroke est composé d'une fusion de un ou plusieurs tronçon du réseau, comme expliqué dans le diagramme 3.3 ci-dessous.

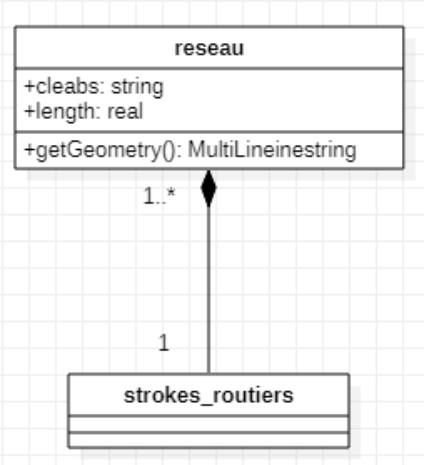


FIGURE 3.3 – Diagramme des classes reseau et strokes_routiers

3.1.3 Création des strokes d'itinéraire

Définition d'un stroke d'itinéraire

Un stroke d'itinéraire est une portion d'itinéraire qui, tout le long de son tracé, partage le chemin avec les mêmes itinéraires.

Parfois un même itinéraire passe deux fois sur une même portion de chemin (dans le cas de boucles ou d'itinéraires en aller-retour par exemple), il faut alors créer à cet endroit deux strokes. Ce choix a été réalisé pour que la prochaine personne travaillant sur ce problème puisse gérer à sa façon les aller-retours.

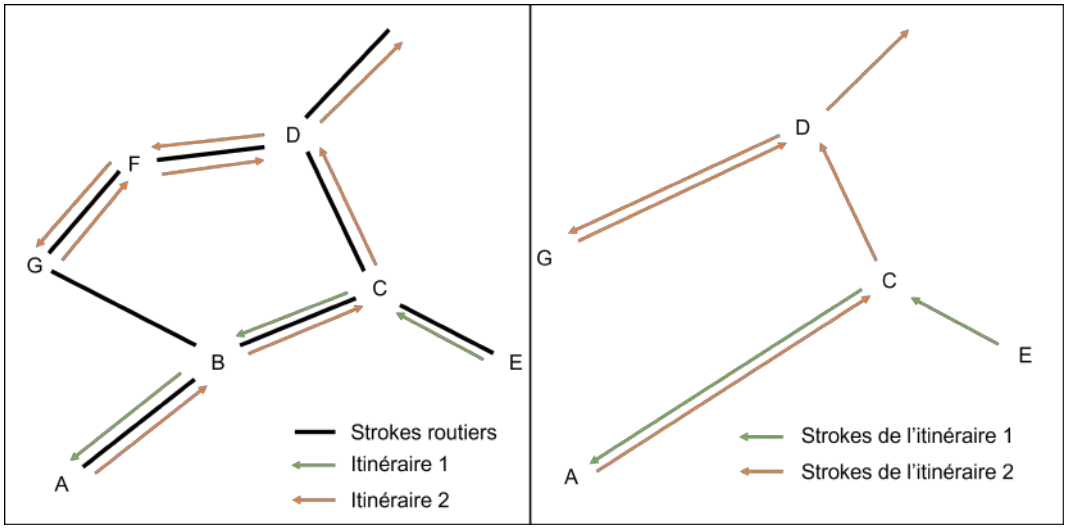


FIGURE 3.4 – Illustration de la définition des strokes d'itinéraires

Dans la figure 3.4, les itinéraires portés par les strokes routiers AB et BC sont fusionnés pour faire les strokes car ces deux portions portent exactement les mêmes itinéraires. De la même façon, les itinéraires portés par les strokes routiers DF et FG sont fusionnés. Deux strokes sont gardés au final pour représenter l'aller-retour fait par l'itinéraire 2.

La figure met en évidence le choix qui a été fait que chaque stroke d'itinéraire ne porte qu'un seul itinéraire et soit caractérisé par son sens. Ce choix se justifie par la volonté de laisser possible le changement des géométries des itinéraires indépendamment. Chaque stroke comporte son sens, qui est utilisé lors de l'affichage : le décalage du symbole se fait suivant le sens du trait.

À cause de cela, il est essentiel que le fichier shapefile des itinéraires donne chaque itinéraire en un unique MultiLineString tracé dans le sens de parcours voulu. Le diagramme de classe ci-dessous montre cette relation.

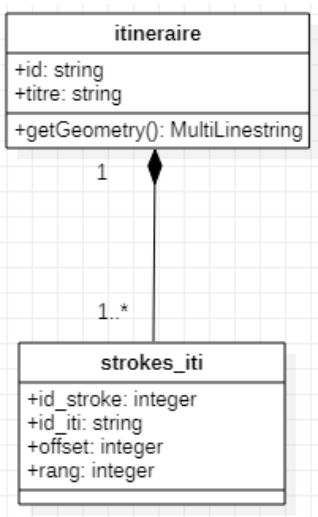


FIGURE 3.5 – Diagramme des classes itineraire et strokes_iti

Méthode

Pour réaliser les strokes d'itinéraires, il faut d'abord diviser chaque itinéraire en segments. Puis, il faut détecter les nœuds principaux du graphe obtenu lorsque :

- le nombre de directions différentes empruntées par les itinéraires pour arriver ou repartir du noeud est de 3 ou plus ;
- ou si un itinéraire effectue un demi-tour en ce point.

Finalement, il faut fusionner les segments d'un même itinéraire du début à la fin en arrêtant la fusion au niveau des nœuds.

3.1.4 Visualisation des itinéraires

Chaque itinéraire est représenté en décalé par rapport au sentier emprunté. Le sentier et les itinéraires sont représentés par la même largeur, largeur que l'utilisateur peut spécifier (comme décrit dans la partie 4.2), et qui est nulle par défaut.
La méthode décrite dans le papier de Teulade-Denantes, Maudet et Duchêne, 2015 est utilisée pour la représentation des itinéraires : à chaque stroke est associé une valeur d'offset (un décalage). Cet offset est un entier relatif qui caractérise la position de l'apparence du stroke par rapport à la position réelle de sa géométrie, en fonction de son sens. Le décalage réel est le produit de cet offset par la largeur de l'apparence. Les itinéraires sont collés entre eux, ce qui rend l'ensemble plus compact et permet un rendu graphique proche des plans de métro ou de bus. Ce décalage peut facilement être modifié dans le code, mais ce n'est pas un élément pris en compte dans le plugin.

Figure 3.6 – Position de l'apparence par rapport à la géométrie (en noir) suivant la valeur d'offset : Teulade-Denantes, Maudet et Duchêne, 2015

Il nous faut donc implémenter un petit programme qui attribut un offset aléatoire à chaque stroke de façon à voir chaque itinéraire passant par chaque stroke. Cet algorithme n'est présent que pour une première visualisation, l'utilisateur pourra ensuite déplacer les itinéraires suivant son envie.
Pour améliorer la visibilité de chaque itinéraire nous attribuons une couleur différente à chacun de manière équirépartie parmi les teintes. Dans un second temps nous laisserons l'utilisateur choisir la couleur qu'il souhaite pour chaque itinéraire.

3.2 Outil de sélection des itinéraires

Le décalage étant géré au niveau de la symbologie des strokes d'itinéraires, sans en modifier réellement la géométrie, les outils natifs de QGIS ne permettent plus à l'utilisateur d'interagir avec les strokes décalés. Afin que l'utilisateur puisse aisément sélectionner un stroke d'itinéraire en cliquant sur sa représentation décalée dans le canevas, il devient nécessaire d'implémenter un nouvel outil de sélection plus adapté.
Cet outil repose sur la possibilité de récupérer les coordonnées terrain de l'emplacement de la souris sur le canevas. A partir de ces coordonnées, on peut calculer la distance

à laquelle se trouve la souris d'un stroke d'itinéraire et en déduire la valeur d'offset à considérer. L'algorithme de gestion du clic se résume en les étapes suivantes :

— Calcul des coordonnées terrain de la souris au clic ;
— Récupération des 10 strokes d'itinéraires les plus proches ;

Pour chaque stroke d'itinéraire récupéré :

— Récupération du côté du clic par rapport au stroke ;
— On calcule $D = (d - offset)$, avec d la distance au stroke et l'offset réel en mètres.

On récupère enfin le stroke qui minimise D et on ne le garde que si la valeur de D est inférieure à la moitié de la largeur du stroke, pour être certain que le clic se situe bien sur le stroke.

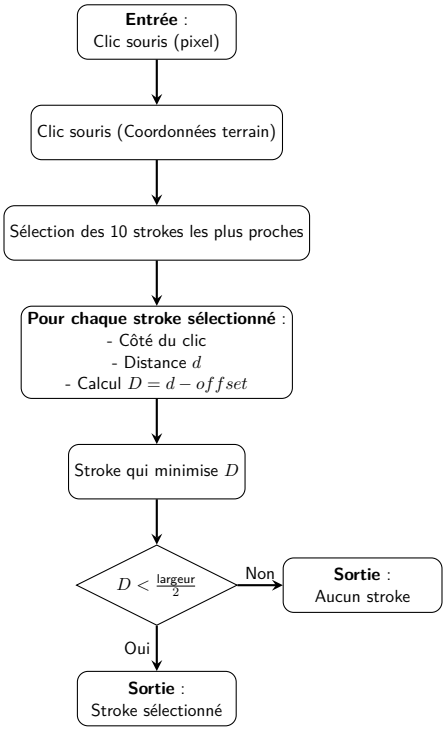


FIGURE 3.7 – Algorithme de sélection de stroke décalé

3.3 Outil d'échange des itinéraires

Cet outil, qui est une suite logique de la sélection des itinéraires, permet à l'utilisateur par l'intermédiaire du plugin d'échanger deux strokes d'itinéraires de la même portion de route. Lorsqu'un itinéraire est sélectionné, il est mémorisé et si un autre est sélectionné (qui n'est pas lui même) alors ils sont intervertis.

Le principe « simple » est d'intervertir les valeurs d'offsets du stroke en mémoire et du stroke sélectionné. Cependant, comme le décalage est calculé en fonction du sens, il est préférable de ne pas échanger exactement ces deux valeurs au risque d'avoir, dans certains cas, des strokes qui se superposent ou qui ne soient plus continus. Il faut donc prendre en compte le sens avant d'intervertir deux strokes, mais il faut aussi prendre en compte le signe d'origine des offsets. Par exemple, si deux segments vont dans le même

sens et que leurs valeurs d'offset ont le même signe, ils sont du même côté de la route et l'échange peut se faire sans problème. Pour cela, on associe les strokes à des vecteurs et la fonction calcule leur produit scalaire.

3.4 Génération des croisements

Un des objectifs de ce projet était de gérer la géométrie des itinéraires au niveau des croisements afin que ces derniers soient les plus naturels et donc aisément compréhensibles par l'utilisateur. En effet, à ce stade, les extrémités des strokes d'itinéraires ne sont pas reliées entre eux.

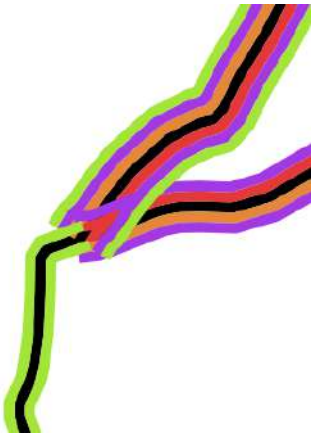


FIGURE 3.8 – Croisements bruts

La méthode que nous avons décidé de mettre en place afin de gérer les intersections est d'arrêter l'apparence des itinéraires à proximité des nœuds constituants les croisements, puis de remplacer l'apparence par une nouvelle respectant certaines conditions et reliant les bonnes extrémités entre elles.
On peut découper la méthode en 5 étapes principales :

- Détecter les nœuds (croisements) ;
- Arrêter l'apparence des itinéraires à une certaine distance des nœuds ;
- Retrouver les nouveaux points finaux (extrémités) des apparences ;
- Relier les points consécutifs entre eux ;
- Réaliser une géométrie permettant une « belle » visualisation des croisements.

Mise en œuvre et résultats

Chapitre 4

En entrée de notre algorithme, nous somme partis des fichiers shapefile donnés pour la réalisation du projet. Nous demandons donc à l'utilisateur futur de fournir :

- Un fichier shapefile du réseau contenant des géométries de type linéaire, sur lequel s'appuient les itinéraires. Les entités du fichier doivent à minima s'arrêter au niveau des croisements. Par exemple, un réseau issu de la BDTOPO convient parfaitement ;
- Un fichier shapefile des itinéraires contenant des géométries de type linéaire. Chaque itinéraire doit avoir au moins un identifiant de type string compris dans un champ nommé « id ». Chaque itinéraire doit être composé d'une seule entité de type polyligne, orientée de son point de départ à son point d'arrivée. Les itinéraires doivent s'appuyer exactement sur la géométrie du réseau, cela veut dire que deux itinéraires passant par une même section de chemin doivent partager les mêmes points de passages que ceux de la géométrie du chemin emprunté.

Ces deux fichiers doivent dépendre du même système de projection.

4.1 Mise en œuvre des méthodes

4.1.1 Création des strokes

Création des strokes routiers

Un géotraitement QGIS existe et permet d'obtenir directement une simplification en strokes d'un graphe planaire. Pour éviter l'apparition d'erreurs lors de la réalisation de ce géotraitement, nous avons décidé d'appliquer une correction préventive des géométries de la couche en entrée en appliquant directement le géotraitement « fixgeometries » à la couche donnée en entrée.
Après cela nous avons appliqué le géotraitement « dissolve » en activant le paramètre « garder les entités disjointes séparées » à la couche ayant la géométrie réparée. Appliquer ce géotraitement à un graphe fusionne directement les entités comprises entre les nœuds de degré supérieur strictement à 2, cela permet donc de créer les strokes routiers.
En résultat, nous obtenons une couche de géométrie multilinestring nommée « strokes _routiers » constituée des strokes des sentiers de randonnée.

Création des strokes d'itinéraires

D'abord, nous avons découpé chaque multilinestring d'itinéraire en segments de deux points, en récupérant un à un les segments composant chaque itinéraire. Chaque segment résultat comprend un champ id, où on retrouve l'identifiant de l'itinéraire dont il est issu,

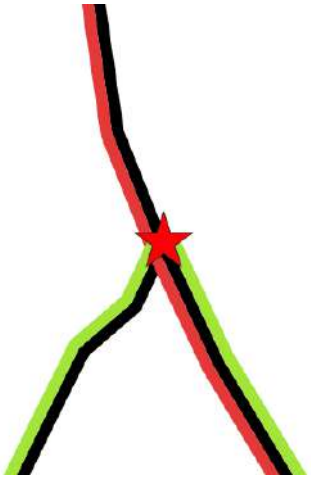
et un champ rang comprenant sa position dans cet itinéraire. Si la longueur du segment est inférieure à un seuil de tolérance, le segment n'est pas gardé.

Nous avons ensuite détecté les nœuds principaux en créant tout d'abord un dictionnaire pt_voisins, qui à chaque point du graphe associe les points qui sont situés à un seul segment (en ne répétant pas les points qui sont superposés).

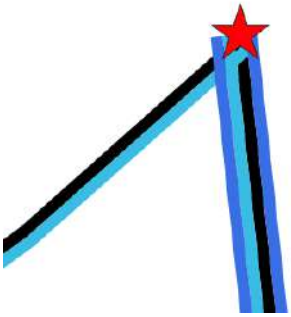
La sélection finale des points repose sur deux critères :

1. **Connectivité multiple :** Pour tout point emprunté par au moins trois chemins de directions différentes, on parcourt la liste des points du graphe. S'il possède strictement plus de deux voisins, le point est conservé.

2. **Points terminaux et demi-tours :** Pour tout point emprunté par un unique chemin ou deux chemins de directions différentes, on regarde si l'itinéraire ne sollicite qu'un seul de ses voisins. Si c'est le cas, il est également retenu. Cette configuration caractérise soit une extrémité de parcours, soit un demi-tour sur le réseau.



(a) Point de degré apparent > 2



(b) Point de degré apparent < 2 mais demi-tour de l'itinéraire bleu foncé

FIGURE 4.1 – Présentation des nœuds principaux du graphe extrait

Nous obtenons ainsi une couche avec les nœuds principaux du graphe. Nous réutiliserons ultérieurement la gestion des croisements, ainsi nous ajoutons un attribut « id » pour identifier chaque nœud.

Une fois les nœuds principaux extraits, nous avons fusionné un à un les segments d'itinéraires en respectant le sens et le rang des segments dans l'itinéraire et en arrêtant la fusion sur chaque nœud principal du graphe. Ainsi est obtenue la couche des strokes d'itinéraires.

Chaque stroke d'itinéraire possède un identifiant unique compris dans son attribut « id_stroke », l'identifiant et le nom de l'itinéraire dont il fait partie dans les attributs « id_iti » et « titre », une valeur représentant son rang parmi les strokes du même itinéraire comprise dans l'attribut « rang » et une valeur d'offset pour son apparence comprise dans le champ « offset ».

4.1.2 Attribution d'un offset pour chaque itinéraire

Pour la visualisation des itinéraires à la manière de plan de bus, nous devons attribuer un offset à chaque stroke d'itinéraire. L'idée étant que deux strokes superposés et allant dans le même sens ne peuvent pas avoir le même offset, alors que s'il vont dans le sens inverse l'un de l'autre ils le peuvent.

Nous parcourons tous les strokes d'itinéraires un à un. Pour chacun :

- Nous créons un dictionnaire Dict_sens_offset dans lequel on retrouve deux listes : une liste des offsets des strokes d'itinéraires ayant la même géométrie et allant dans le **même sens**, et une liste des offsets des strokes d'itinéraire ayant la même géométrie mais un **sens différent**.
- Nous attribuons une valeur d'offset au stroke considéré, en regardant la valeur absolue maximale M prise par les offsets du dictionnaire Dict_sens_offset :
  - Si M est atteinte en valeur absolue dans le dictionnaire alors : offset = M+1 ;
  - Si M est atteinte en valeur absolue dans la liste des offsets de même sens alors on récupère la valeur signée de M (qu'on appelle SM) puis : offset = -SM ;
  - Si M est atteinte en valeur absolue dans la liste des offsets de sens opposé alors on récupère la valeur signée de M (qu'on appelle SM) puis : offset = SM.

FIGURE 4.2 – Visualisation des offset suivant le sens des itinéraires

4.1.3 Outil de sélection

Afin d'implémenter un tel outil avec PyQGIS, il est nécessaire de travailler en orienté objet (classes, méthodes...) et de créer une classe spécifique à la sélection d'itinéraires décalés. La classe créée étend QgsMapTool, qui constitue l'interface standard de QGIS pour les outils de carte. Le diagramme ci-dessous détaille les outils utilisés afin de mettre en œuvre la méthode détaillée précédemment :

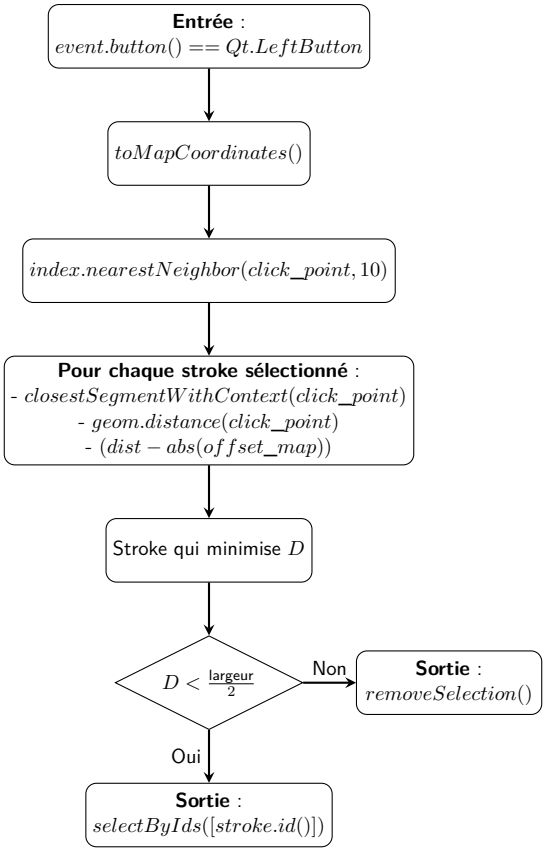


FIGURE 4.3 – Implémentation de l'algorithme de clic en PyQGIS

L'outil est ensuite intégré au plugin, qui permet d'activer ou de désactiver cette méthode de sélection.

4.1.4 Outil d'échange

Pour l'échange, la sélection du premier itinéraire fonctionne comme l'outil de sélection décrit plus haut, si ce n'est que, pour un itinéraire sélectionné, son vecteur de direction est également calculé. Ceci s'effectue grâce aux coordonnées de ses deux sommets, et les deux sont sauvegardés en paire. Si l'utilisateur clique sur un itinéraire différent, l'échange peut se réaliser. Si le clic se fait sur le même itinéraire ou dans le vide, la sélection est annulée.

Pour prendre en compte la direction des strokes, on calcule le produit vectoriel des deux strokes sélectionnés. C'est pour cela qu'on calcule le vecteur lors de la sélection. Ensuite, une fois la direction connue, il faut regarder dans chacun des deux cas le signe des deux offsets. La matrice de décision met en évidence les quatre situations possibles et les solutions envisagées :

	Même direction	Directions inverses
Signe égal	Les segments sont du même côté : pas de problème.	Les segments sont de part et d'autre de la route : pas de problème.
Signe différent	Les segments sont de part et d'autre de la route : il faut multiplier par -1 les valeurs d'offset.	Les segments sont du même côté : il faut inverser les valeurs absolues et garder le signe original de l'offset.

TABLE 4.1 – Matrice de décision

4.1.5 Génération des croisements

L'implémentation de l'algorithme de création des croisements a été réalisée via PyQGIS.

La première étape consiste à récupérer chaque croisement issu de l'algorithme de création des strokes, à effectuer un buffer autour de ces derniers grâce au géotraitement natif $buffer$, puis à en extraire la frontière (géotraitement $boundary$). La taille du buffer a, pour le moment, été choisie de manière unique pour tout le réseau (5 m). Elle devra dépendre, à terme, de la structure de celui-ci (proximité des croisements, angle de convergence de plusieurs itinéraires, degré du croisement, etc.)

On calcule ensuite les points résultant de l'intersection entre les strokes des itinéraires et la frontière du buffer grâce au géotraitement $lineintersections$. Les points ainsi obtenus sont superposés lorsque les strokes des itinéraires partagent le même stroke routier. En effet, l'intersection ne se fait pas sur le décalage, mais sur la géométrie réelle des strokes d'itinéraires. Ces points doivent récupérer les attributs correspondant à l'identifiant du croisement ainsi que les attributs permettant d'identifier le stroke (id_stroke, $itin_id$, $id_croisement$, $rang$, $offset$).

Pour que le choix des points à relier entre eux puisse se faire, il est nécessaire de stocker un nouvel attribut $extremite$ pour chaque point résultant de cette intersection, valant 1 si le point correspond au début du stroke d'itinéraire et 0 s'il correspond à son extrémité finale (selon le sens d'enregistrement de la géométrie). Afin de récupérer cette valeur, on tronque la géométrie des strokes des itinéraires selon le même buffer (géotraitement $difference$) et on compare chaque point d'intersection avec les points de départ et d'arrivée du stroke considéré. Le point pour lequel cette distance est minimale indique de quel côté du stroke se situe le point d'intersection.

L'étape suivante consiste à calculer les coordonnées du point décalé selon l'offset du stroke d'itinéraire. Pour cela, on calcule la normale à l'itinéraire au voisinage du point à l'aide des fonctions d'interpolation le long d'une ligne. Puis, grâce à l'attribut $offset$, on calcule le décalage nécessaire le long de la normale. Le nouveau point obtenu correspond ainsi à la nouvelle extrémité du stroke d'itinéraire décalé.

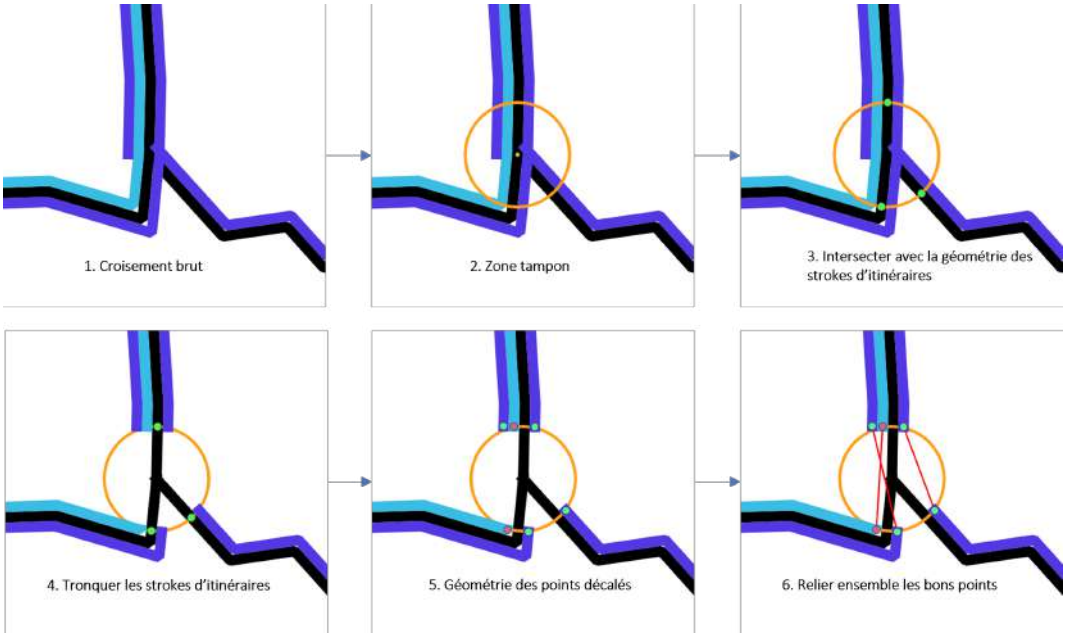


FIGURE 4.4 – Étapes pour la gestion des croisements

La dernière étape consiste enfin à relier ces points par une certaine géométrie. Pour le moment, la géométrie réalisée est restée sommaire et a vocation a être améliorée, car elle ne permet absolument pas une bonne visualisation des intersections. Si l'on considère un Point A et un Point B, on peut les relier s'ils vérifient ensemble les conditions suivantes :

— A et B ont les mêmes $id_croisement$ et $id_itinéraire$;
— $rang(B) = rang(A) + 1$, avec $rang$ le rang du stroke dans l'itinéraire ;
— $extremite$ de A vaut 0 et $extremite$ de B vaut 1 (i.e. A est une arrivée et B un départ de stroke).

Les couples de points sont ensuite reliés entre eux en créant une ligne droite.

La dernière étape de la méthodologie (création d'une « belle » géométrie) n'a pas pû être implémentée.

4.2 Interface graphique et fonctionnalités

Création du plugin

Le plugin Itinéraires Décalés a été créé à l'aide du plugin QGIS $Plugin\ Builder$. L'interface utilisateur a été réalisée grâce à l'outil de design d'UI de QGIS, Qt Designer. Toutes les fonctionnalités sont d'abord imaginées et représentées via l'UI, puis l'UI est convertie en code Python.

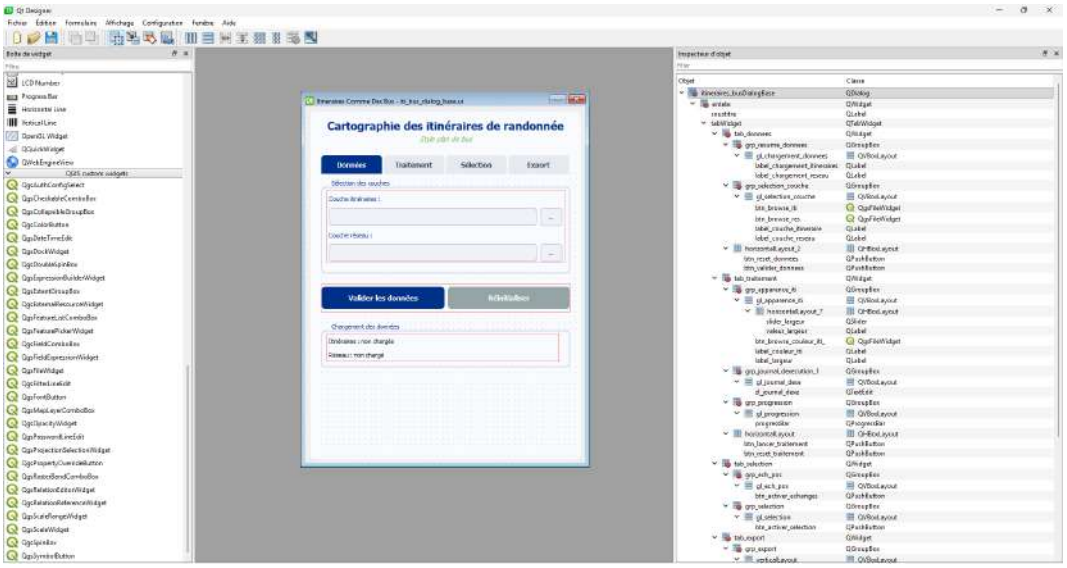


FIGURE 4.5 – Interface de Qt Designer : tous les Widgets (à droite) ont un nom unique qui permet de les relier à une fonctionnalité précise.

Le fichier `itineraires_decales_dialog.py` joue un rôle central : il relie les éléments de l'interface aux fonctions du programme.
Le plugin se présente sous la forme d'une interface simple et intuitive, organisée en quatre onglets distincts. Ces onglets guident l'utilisateur à travers les différentes étapes du traitement et les fonctionnalités présentes.

Onglet Données

Cet onglet est le point de départ du plugin. Il permet à l'utilisateur d'importer les deux couches vectorielles des itinéraires et du réseau routier.
Une fois les couches sélectionnées, l'utilisateur clique sur « Valider les données ». Le bouton « Réinitialiser » nettoie les couches chargées.
En bas de l'onglet, une zone de statut affiche si les couches ont été chargées ou non.

Onglet Traitement

Cet onglet permet à la fois de régler l'apparence des itinéraires et de lancer le traitement. Il est divisé en deux parties principales.
Le groupe « Apparence des itinéraires » permet de personnaliser la couleur et l'épaisseur des tracés des itinéraires. L'utilisateur peut charger un fichier externe pour définir la couleur de chaque itinéraire. Sinon, des couleurs sont assignées aléatoirement aux itinéraires. L'utilisateur peut également régler l'épaisseur des tracés avec le curseur.
Le bouton « Lancer tous les traitements » exécute automatiquement l'ensemble des opérations liées à la création des strokes. Le bouton « Réinitialiser » permet de revenir aux paramètres initiaux.
La barre de progression se remplit au fur et à mesure des traitements et permet à l'utilisateur de se rendre compte que le traitement est encore en cours. Les différentes étapes s'affichent dans le journal, ainsi que les éventuelles erreurs.

Onglet Sélection

Cet onglet permet à l'utilisateur d'interagir directement avec les itinéraires affichés sur la carte QGIS.

Le bouton « Activer la sélection par click » active l'outil de sélection. Une fois activé, l'utilisateur peut cliquer sur un ruban coloré dans la carte pour le sélectionner. Le plugin identifie automatiquement le stroke le plus proche du point cliqué, comme décrit dans la section 3.2.

Le bouton « Activer la phase d'échange de strokes » active l'outil d'échange des strokes. Une fois activé, l'utilisateur peut sélectionner un premier et/ou un deuxième stroke, ce qui procède à leur échange s'ils appartiennent au même segment routier. Cette fonctionnalité permet d'ajuster manuellement l'affichage des itinéraires, en complément du placement automatique.

Onglet Export

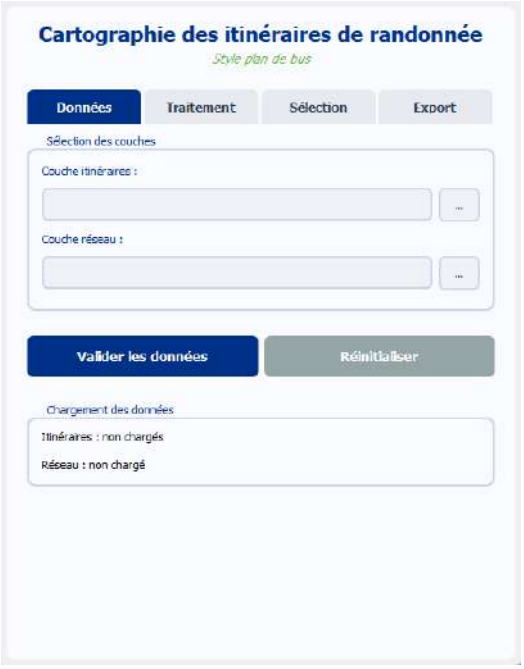
Ce dernier onglet permet à l'utilisateur d'exporter les résultats du traitement dans plusieurs formats.

L'utilisateur peut sélectionner le format qui lui convient parmi quatre options : *Shapefile*, *GeoPackage*, *PNG* (carte) et *PDF* (carte). Les deux premiers permettent d'exporter les données afin de les réutiliser dans un autre logiciel SIG, tandis que les deux derniers permettent d'exporter une image de la carte telle qu'elle apparaît dans QGIS, pour le partage ou l'impression.

L'utilisateur sélectionne l'emplacement où seront enregistrés les fichiers exportés.

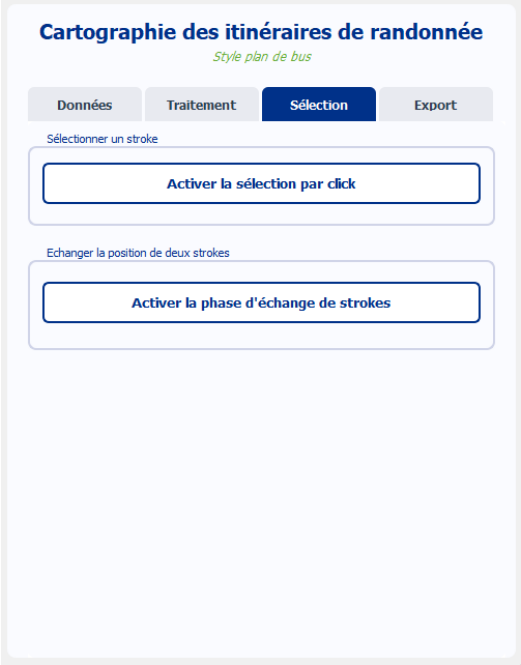
Le bouton « Exporter » permet de démarrer l'opération, tandis que le bouton « Réinitialiser » permet d'annuler et de réinitialiser les paramètres.

Enfin, un journal situé en bas de l'onglet affiche les différentes étapes de l'export et les éventuelles erreurs.

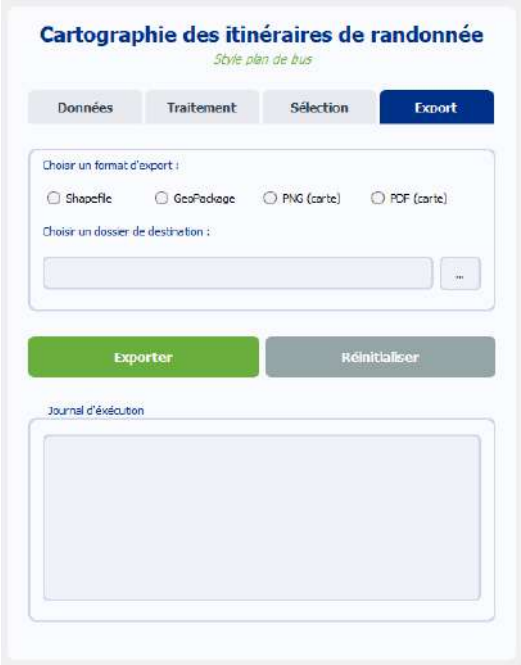


(a) Onglet Données

(b) Onglet Traitement



(c) Onglet Sélection



(d) Onglet Export

FIGURE 4.6 – Les quatre onglets du plugin

4.3 Limites et perspectives

4.3.1 Limites et difficultés rencontrées

Le plugin développé permet de représenter les itinéraires superposés de façon plus lisible dans QGIS, grâce à la création de strokes, au calcul des décalages (offsets) et à un outil de sélection adapté. Cependant, certaines limites existent.

La gestion des croisements reste le principal problème du projet, notamment en raison de la grande quantité de cas à traiter. Les raisons pour lesquelles cette gestion est difficile sont tout d'abord liées à la grande diversité des croisements. Certains croisements auront des routes qui convergent lentement (avec un angle très petit) et d'autres avec un angle très grand. Dès lors, il devient extrêmement compliqué de gérer l'affichage à ces niveaux, surtout si le nombre d'itinéraires incidents au croisement est élevé. Une autre difficulté provient de la proximité des croisements. En effet, certains croisements sont très proches, il suffit alors qu'il y ait un nombre d'itinéraires un peu trop important pour rendre la gestion de ces croisements très complexe. On pourrait songer à rassembler certains croisements successifs très proches (cf fig 4.7), mais cela ne résoudrait pas le problème des croisements très proches mais non connectés entre eux (par exemple dans les zones de très forte pente où l'on peut avoir un croisement en haut d'une falaise et un en bas, cf fig 4.7, le croisement de gauche intersecte un chemin plus bas et coupe ses itinéraires).

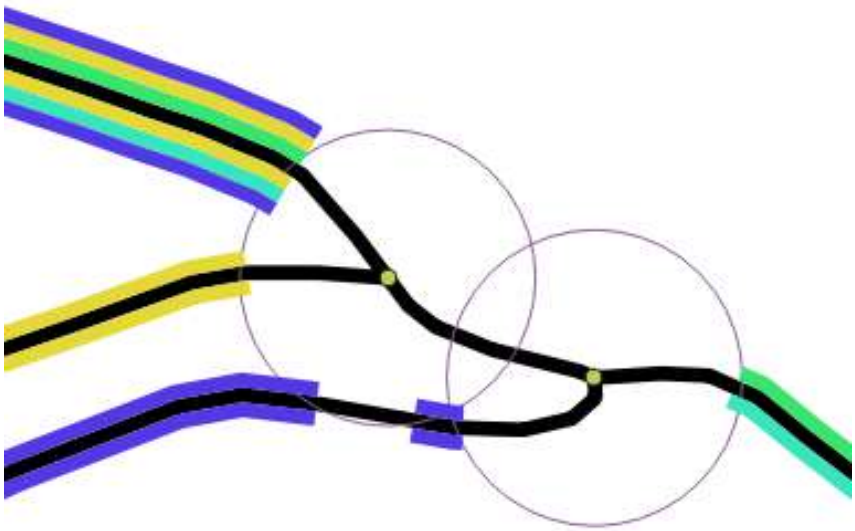


FIGURE 4.7 – Limite de la visualisation des croisements

Par ailleurs, le fonctionnement du plugin dépend de la qualité des données d'entrée. Si les géométries sont mal structurées, les itinéraires sont mal orientés ou s'il y a des erreurs topologiques, cela peut poser des problèmes lors de la création des strokes et de l'affichage final.

Les principales difficultés auxquelles nous avons été confrontés sont liées à la découverte

de cas de plus en plus particuliers suivant l'avancement dans le projet. Par exemple, le souhait de garder l'information lorsque les itinéraires réalisent des aller-retours nous a obligé à diviser les itinéraires en segments puis à les refusionner pour la création des strokes. Aussi cela nous a-il obligé à créer des cas particuliers de détection de noeuds, notamment lorsque des itinéraires font demi-tour sur des noeuds de degré apparent inférieur à 3. Une des faiblesses de notre algorithme est qu'il est donc adapté aux cas particuliers rencontrés sur le jeu de données test mais qu'il n'est sans doute pas adapté pour d'autres cas particuliers non rencontrés sur ce jeu de données.

4.3.2 Perspectives d'amélioration

Dans la continuité de ce travail, plusieurs perspectives d'amélioration peuvent être envisagées afin d'enrichir les fonctionnalités du plugin et d'améliorer la qualité des résultats.

Tout d'abord, améliorer la gestion des croisements est une priorité. L'objectif est d'obtenir des raccords plus fluides, plus naturels et plus lisibles entre les différents strokes, surtout dans les zones complexes. Pour cela, il est nécessaire de travailler sur une géométrie optimale permettant de relier les points correspondant aux extrémités des strokes d'itinéraires. Cette géométrie pourrait par exemple s'appuyer sur des courbes de Bézier, qui sont une manière de tracer des courbes contraintes par un certain nombre de points. Une fois les croisements correctement générés, il faut qu'ils soient recalculés après que l'utilisateur ait décidé de changer la disposition des itinéraires.

L'algorithme de détection des nœuds principaux devrait prendre en entrée la tolérance qui permet de supprimer les trop petits segments afin d'éviter l'omission de certains nœuds lors du listage des voisins.

L'algorithme de fusion des segments ne les fusionne pas vraiment dans le sens de rang croissant pour l'instant, la correction de ce problème règlera des coquilles apparaissant encore dans la création des strokes d'itinéraire.

Concernant l'outil de sélection, il ne peut pas en l'état fonctionner au niveau des croisements. En effet, au voisinage des intersections, les géométries des itinéraires n'ont pas un décalage symbologique, mais réel. De plus, en cas de superposition de plusieurs itinéraires sur une zone avec de forts lacets, il apparaît un espace vide au niveau des angles et la géométrie est légèrement plus décalée que ce qui est prévu par l'offset, ce qui fausse le calcul réalisé par l'outil de sélection et engendre un léger décalage entre la zone de clic et la visualisation de l'itinéraire.

L'outil de sélection devrait également contenir un algorithme permettant d'évaluer le nombre de strokes d'itinéraire à récupérer en fonction du nombre maximal d'itinéraires passant par un même chemin afin de rendre l'outil plus flexible.

Il serait également pertinent d'améliorer l'interface du plugin en proposant des options de personnalisation, notamment pour les couleurs, ainsi que des fonctionnalités permettant d'ajouter, de modifier ou de supprimer des itinéraires de manière simple et flexible.

Finalement, la création et l'évaluation des algorithmes et du plugin qui en découle s'est limitée à la zone d'étude et aux données présentées. Ainsi, il serait pertinent de tester les fonctionnalités du plugin sur d'autres territoires et sur d'autres types de réseaux afin d'évaluer sa robustesse et son potentiel de diffusion dans des contextes variés.

Conclusion

En conclusion, l'objectif du projet était de concevoir un outil interactif sous QGIS permettant de visualiser des itinéraires « à la manière des plans de bus », tout en offrant à l'utilisateur la possibilité de sélectionner et de modifier leur disposition à l'aide d'outils de sélection. Le travail réalisé a abouti à la création d'un plugin QGIS doté d'une interface graphique. Celui-ci permet, à partir d'un réseau routier et d'un réseau d'itinéraires correctement formatés, de générer une visualisation des itinéraires décalés, colorés, et interactifs, avec la possibilité de sélectionner et d'échanger des portions d'itinéraires. Concernant la gestion des croisements, un algorithme (actuellement non intégré au plugin) a été développé afin de tronquer les itinéraires à proximité des intersections et de les relier de manière simplifiée à l'aide de segments droits. Toutefois, des améliorations restent nécessaires pour rendre les algorithmes encore plus robustes face aux nombreux cas particuliers rencontrés. Enfin, la génération et l'adaptation des croisements constituent encore une piste de réflexion importante, afin de rendre la visualisation des réseaux d'itinéraires la plus cohérente.

Bibliographie

Bast, H., P. Brosi et S. Storandt (2019). "Efficient Generation of Geographically Accurate Transit Maps". In : *ACM Transactions on Spatial Algorithms and Systems* Vol 5.4, p. 1-36. doi : 10.1145/33377906.

Nöllenburg, M. (2014). *A survey on automated metro map layout methods*. Rapp. tech. Department of Informatics, Karlsruhe Institute of Technology (KIT).

Nöllenburg, M. et A. Wolff (2011). "Drawing and labeling high-quality metro maps by mixed-integer programming". In : *IEEE Transactions on Visualization and Computer Graphics* Vol 17.5, p. 626-641. doi : 10.1109/TVCG.2010.81.

Teulade-Denantes, J., A. Maudet et C. Duchêne (2015). "Routes visualization : Automated placement of multiple route symbols along a physical network infrastructure". In : *Journal of Spatial Information Science (JOSIS)* Vol 132. doi : 10.5311/JOSIS.2015.11.230.

Thomson, R. et D. Richardson (1999). "The "Good Continuation" Principle of Perceptual Organization Applied to the Generalization of Road Networks". In : *Proceedings of the 19th International Cartographic Conference (ICC)*. Ottawa, Canada.

Xiao, T. et al. (2024). "A road generalization method using graph convolutional network based on mesh-line structure unit". In : *Geocarto International* Vol 39.1. doi : 10.1080/10106049.2024.2413549.

Table des figures

1 Exemple d'un plan de bus : Plan du réseau de bus RATP de Paris (Source : RATP) 5

2.1 Localisation de la zone d'étude dans le massif du Vercors. 13
2.2 Visualisation de l'emprise des données dans la zone d'étude 14

3.1 (a) Modélisation initiale : 7 strokes de route et 7 segments d'itinéraire ; (b) Segments d'itinéraires regroupés en strokes : 4 strokes d'itinéraire créés à partir des 7 segments. Source : Teulade-Denantes, Maudet et Duchêne, 2015 16
3.2 Visualisation des strokes routiers : une couleur par stroke. 17
3.3 Diagramme des classes reseau et strokes_routiers 17
3.4 Illustration de la définition des strokes d'itinéraires 18
3.5 Diagramme des classes itineraire et strokes_iti 18
3.6 Position de l'apparence par rapport à la géométrie (en noir) suivant la valeur d'offset : Teulade-Denantes, Maudet et Duchêne, 2015 . 19
3.7 Algorithme de sélection de stroke décalé 20
3.8 Croisements bruts 21

4.1 Présentation des nœuds principaux du graphe extrait 23
4.2 Visualisation des offset suivant le sens des itinéraires 24
4.3 Implémentation de l'algorithme de clic en PyQGIS 25
4.4 Étapes pour la gestion des croisements 27
4.5 Interface de Qt Designer : tous les Widgets (à droite) ont un nom unique qui permet de les relier à une fonctionnalité précise. 28
4.6 Les quatre onglets du plugin 30
4.7 Limite de la visualisation des croisements 31

Liste des tableaux

4.1 Matrice de décision 26